

Model Format Conversion

Model Format Conversion

1. Model Format Introduction
 - Common Model Formats
2. Course Content
3. Model Conversion
 - 3.1 Model Conversion Program
 - 3.1 Start Conversion
4. Compare and Verify Acceleration Effect of Exported Model
5. Source Code Analysis
 - 6.1 Model Conversion Program

[!IMPORTANT]

- The factory image already contains converted format traffic sign and lane detection models. This section tutorial is provided for users who need to master model conversion methods.
- If you want to experience the complete autonomous driving functionality directly, you can skip this section!!!

1. Model Format Introduction

- The ultimate goal of training models is to deploy them in practical applications. The export mode in YOLOv11 provides multiple options to export trained models in different formats, making them deployable on various platforms and devices.



Common Model Formats

- **PyTorch (.pt)**: Default native format, containing complete model structure and weights.
Use cases: Model training, fine-tuning, foundation for subsequent format export, not suitable for direct deployment.
- **TorchScript (.torchscript)**: Serialized format of PyTorch models, stripping Python dependencies.
Use cases: Production environment deployment in PyTorch ecosystem (such as server-side inference), supports C++ calls.

- ONNX (.onnx): Open Neural Network Exchange format, core standard for cross-framework compatibility.
Use cases: CPU/GPU universal deployment, combined with ONNX Runtime can achieve 3x CPU acceleration.
- TensorRT (.engine): NVIDIA exclusive optimized format, GPU-based inference engine.
Use cases: Real-time inference on NVIDIA GPU (such as Jetson development boards), can achieve 5x GPU acceleration.
- OpenVINO (openvino_model): Intel exclusive format, optimized for Intel CPU/GPU/VPU.
Use cases: Intel hardware (such as PC, edge computing devices) deployment.
- TF Lite (.tflite): TensorFlow lightweight format, targeting mobile/edge devices.
Use cases: Android devices, embedded hardware (such as Raspberry Pi), supports INT8 quantization compression.
- RKNN (rknn_model): Rockchip exclusive format, adapted for its AI chips.
Use cases: Rockchip RK series chips (such as development boards, smart hardware) deployment.
- Comparison of Different Format Models

Comparison	Universal Format (ONNX)	Accelerated Format (TensorRT)	Edge Format (TF Lite)	Native Format (.pt)
Inference Speed	Medium (CPU/GPU Universal)	Extremely Fast (GPU Exclusive)	Slow - Medium (Lightweight)	Slow (Python Dependency)
Model Size	Medium	Medium (Supports Quantization Compression)	Extremely Small (INT8 Quantization)	Larger
Hardware Dependency	None	NVIDIA GPU	Mobile / Embedded Devices	PyTorch Environment
Use Cases	Universal Deployment, Cross-Framework	Real-time High-Concurrency Inference	Low-Power Edge Devices	Training / Fine-Tuning
Precision Support	FP32/FP16	FP32/FP16/INT8	FP32/FP16/INT8	Full Precision

2. Course Content

1. Convert pt model (native format exported by yolov11) to TensorRT format model and compare the inference speed differences between the two

What are the benefits of using TensorRT to export models?

Using TensorRT to export models on the Jetson platform can significantly improve model inference performance. YOLO11 models exported to TensorRT can achieve up to 5x GPU acceleration.

- **Speed:** Achieve faster inference through advanced optimization.
- **Compatibility:** Smooth integration with NVIDIA hardware.

3. Model Conversion

3.1 Model Conversion Program

Model format conversion program path:

```
/home/jetson/yahboomM3_ws/yolo_train/conver_model.py
```

Optional startup parameter descriptions:

- **format:** Model format to export, default is engine (TensorRT format)
- **MODEL_PATH:** Original pt format model path
- **data:** Calibration dataset, only needed when performing int8 quantization on the model
- **device:** Running device number, default is 0
- **int8:** Whether to use int8 quantization, further compressing model size, default is False

3.1 Start Conversion

```
cd ~/yahboomM3_ws/yolo_train
```

- Here we use the traffic sign recognition model trained in the previous section as an example for conversion (replace --MODEL_PATH path with your actual .pt model path):

```
python3 conver_model.py --MODEL_PATH  
/home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.pt
```

- After model conversion and export is completed, the terminal prompts `Export complete` and the exported model path

```
jetson@yahboom: ~/yahboomM3_ws/yolo_train
jetson@yahboom: ~/yahboomM3_ws/yolo_train 80x33
TensorRT: output "output0" with shape(1, 18, 8400) DataType.FLOAT
TensorRT: building FP32 engine as /home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.engine
[11/18/2025-20:48:12] [TRT] [I] Local timing cache in use. Profiling results in this builder pass will not be stored.
[11/18/2025-20:49:43] [TRT] [I] Compiler backend is used during engine build.
[11/18/2025-20:51:18] [TRT] [I] Detected 1 inputs and 1 output network tensors.
[11/18/2025-20:51:20] [TRT] [I] Total Host Persistent Memory: 543408 bytes
[11/18/2025-20:51:20] [TRT] [I] Total Device Persistent Memory: 0 bytes
[11/18/2025-20:51:20] [TRT] [I] Max Scratch Memory: 2764800 bytes
[11/18/2025-20:51:20] [TRT] [I] [BlockAssignment] Started assigning block shifts. This will take 220 steps to complete.
[11/18/2025-20:51:20] [TRT] [I] [BlockAssignment] Algorithm ShiftNTopDown took 20.5218ms to assign 10 blocks to 220 nodes requiring 19251712 bytes.
[11/18/2025-20:51:20] [TRT] [I] Total Activation Memory: 19251200 bytes
[11/18/2025-20:51:20] [TRT] [I] Total Weights Memory: 10469444 bytes
[11/18/2025-20:51:20] [TRT] [I] Compiler backend is used during engine execution.
[11/18/2025-20:51:20] [TRT] [I] Engine generation completed in 188.427 seconds.
[11/18/2025-20:51:20] [TRT] [I] [MemUsageStats] Peak memory usage of TRT CPU/GPU memory allocators: CPU 1 MiB, GPU 145 MiB
TensorRT: export success 196.7s, saved as '/home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.engine' (12.0 MB)
Export complete (198.6s)
Results saved to /home/jetson/yahboomM3_ws/yolo_train/project/train3/weights
Predict: yolo predict task=detect model=/home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.engine imgsz=640
Validate: yolo val task=detect model=/home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.engine imgsz=640 data=/home/jetson/yahboomM3_ws/yolo_train/train_sign/sign.yaml
Visualize: https://netron.app
jetson@yahboom:~/yahboomM3_ws/yolo_train$
```

- The best.engine in the output directory is the exported TensorRT format model file

```
jetson@yahboom: ~/yahboomM3_ws/yolo_train/project/train3/weights
jetson@yahboom: ~/yahboomM3_ws/yolo_train/project/train3/weights 95x33
jetson@yahboom:~/yahboomM3_ws/yolo_train/project/train3/weights$ ls
best.engine best.onnx best.pt last.pt
jetson@yahboom:~/yahboomM3_ws/yolo_train/project/train3/weights$
```

[!WARNING]

- Note!!! TensorRT format models exported on different GPU platforms are not universal due to different TensorRT versions.
- For example, TensorRT format models exported on jetson orin NX cannot be used on jetson nano because the TensorRT versions on the two platforms are different.

4. Compare and Verify Acceleration Effect of Exported Model

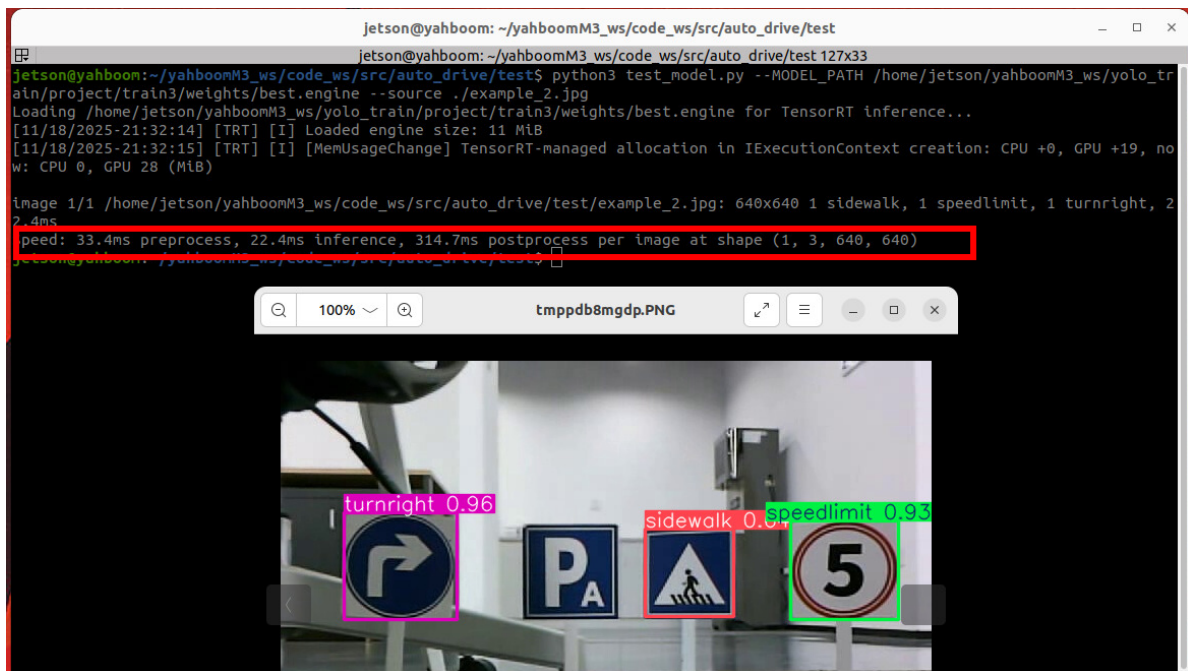
- We can use the test model program from the previous lesson to verify whether the exported model runs normally, and compare the inference speed before and after conversion

```
cd ~/yahboomM3_ws/code_ws/src/auto_drive/test
```

- Use the converted `best.engine` model for inference (replace `--MODEL_PATH` path with your actual `.pt` model path):

```
python3 test_model.py --MODEL_PATH
/home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.engine --source
./example_2.jpg
```

- You can see that the TensorRT format model inference time is 22.4ms (test machine is orin nano)

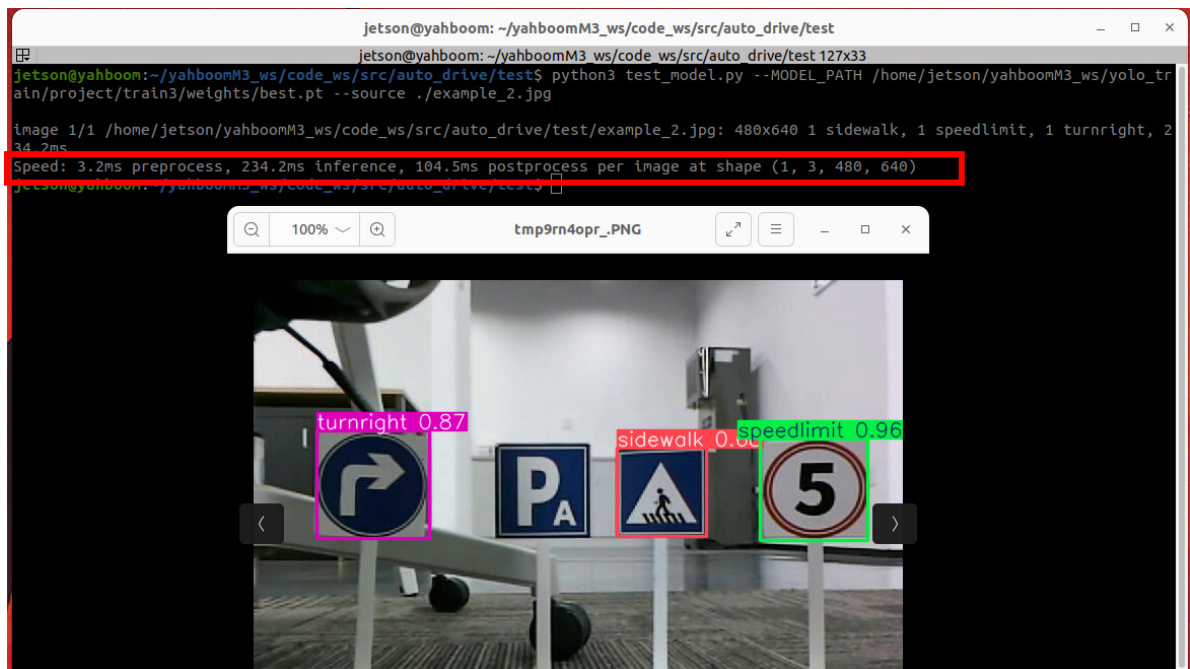


- Then we compare the inference speed of the original `.pt` model exported by YOLOv11

```
python3 test_model.py --MODEL_PATH
/home/jetson/yahboomM3_ws/yolo_train/project/train3/weights/best.pt --source
./example_2.jpg
```

- You can see that the `.pt` model inference time is 234.2ms

Conclusion: TensorRT format model inference speed is more than 10 times faster than PyTorch format model inference speed (test machine is orin nano).



5. Source Code Analysis

6.1 Model Conversion Program

The program source code is located at

`/home/jetson/yahboomM3_ws/yolo_train/conver_model.py`

The program process is divided into:

- Parse parameters: `args = get_args()` reads parameters passed from the command line, such as model path, export format, etc.
- Load model: `model = YOLO(args.MODEL_PATH)` loads YOLO pretrained model from the specified path.
- Export model: Call different export configurations based on whether INT8 quantization is enabled

```
import argparse
from ultralytics import YOLO

def get_args():
    '''
    '''

    parser = argparse.ArgumentParser()
    parser.add_argument('--project', default='./project', help='Project file
output directory', type=str)
    parser.add_argument('--
MODEL_PATH', default='/home/jetson/MODELS/yolov11/yolo11n.pt', help='原始模型权重文
件路径Original model weight file path', type=str)
    parser.add_argument('--epochs', default=100, help='训练轮次Training rounds',
type=int)
    parser.add_argument('--imgsz', default=640, help='图像大小Image size',
type=int)
    parser.add_argument('--train_config', default='', help='配置文件路径
Configuration file path', type=str)
    return parser.parse_args()
```

```
if __name__ == "__main__":  
    args = get_args()  
    model = YOLO(args.MODEL_PATH) # Load a pretrained model  
  
    results = model.train(data=args.train_config, epochs=args.epochs,  
imgsz=args.imgsz,project=args.project)#Start training
```